

Кафедра
статистики и
кибернетики
*Department of statistics and
cybernetics*

Лекция №4 «Хэширование»

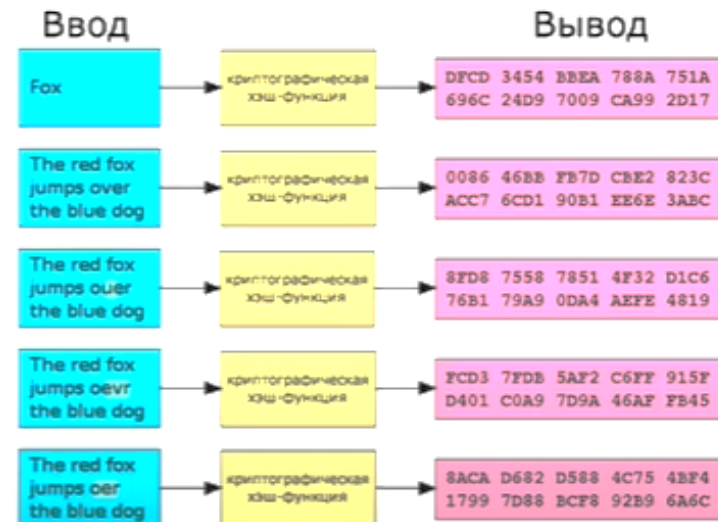
Лектор: Вадим Владимирович Демичев
доцент, кандидат экономических наук

Москва, 2026

Хэширование

- Хэш-функция, чаще называемая просто хешем, — это математический алгоритм, преобразовывающий произвольный массив данных в состоящую из букв и цифр строку фиксированной длины.

- хэш подобен **отпечатку пальца человека**. Как известно, отпечаток пальца уникален и в природе не существует людей с одинаковыми отпечатками. Даже у близнецов отпечатки пальцев разные.



Отличие от шифрования

Хеширование - это односторонняя функция

Шифрование является правильной (двухсторонней) функцией. Это обратимо, вы можете расшифровать искаженную строку, чтобы получить исходную строку, если у вас есть ключ.

Шифрование преобразует какие-либо данные в серию нечитабельных людскому глазу знаков, которые не имеют фиксированной длины.

Основное отличие между шифрованием и хешированием — то, что зашифрованные последовательности могут быть повернуты назад в их оригинальную расшифрованную форму, если конечно соответствующий ключ имеется.

Пример

```
1 def my_hash(s):  
2     return len(s)/10  
3 print(my_hash("ghdgrtaassg"))  
4
```

```
In [1]: runfile('C:/Users/vadid/untitled0.py', wdir='C:/Users/vadid')  
1.1  
  
In [2]:
```

Наиболее известные алгоритмы хеширования

SHA-256 представляет собой однонаправленную функцию для создания цифровых отпечатков фиксированной длины (256 бит, 32 байт) из входных данных размером до 2,31 эксабайт (2^{64} бит) и является частным случаем алгоритма из семейства криптографических алгоритмов SHA-2 (Secure Hash Algorithm Version 2) опубликованным АНБ США в 2002 году.

MD5 (англ. Message Digest 5) — 128-битный алгоритм хеширования, разработанный профессором Рональдом Л. Ривестом из Массачусетского технологического института (Massachusetts Institute of Technology, MIT) в 1991 году. Предназначен для создания «отпечатков» или дайджестов сообщения произвольной длины и последующей проверки их подлинности. Широко применялся для проверки целостности информации и хранения хешей паролей.

Требования к хэш-функции

Каждому набору (массиву) информации присущ строго определенный, **уникальный хэш**.

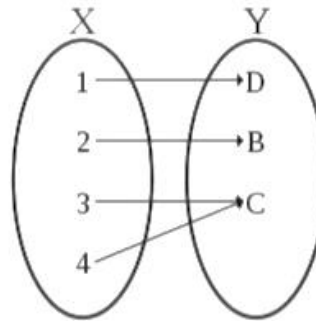
При самом незначительном изменении входной информации её хэш меняется **кардинально**.

Не существует обратной функции, которая из хэша может **восстановить исходный массив** информации.

Хэширование позволяет достаточно **быстро** вычислить искомый хэш для весьма большого массива входной информации.

Коллизии

- Коллизия хеш-функции — два различных входных блока данных x и y для хеш-функции H таких, что $H(x)=H(y)$.
- Математики-криптографы стараются создать такие хэш-функции, вероятность коллизий в которых стремилась бы к нулю.



Задача

- Имеет 1 000 000 файлов, средний объем одного файла 1Гб. Найти дубликаты документов.

Хэширование

- Хэш используется для того, чтобы быстрее отличать одни данные от других без необходимости сравнивать каждый-каждый бит этих данных.
- Достаточно обработать эти данные один раз (вычислить их хэши) и можно сравнивать только их, а это гораздо быстрее.
- Идея такая. Если хэши различаются, значит, это совершенно точно разные данные.
- Если хэши одинаковы, значит, с вероятностью в 99,99999... (и ещё 70 девяток, если предположить идеальное распределение 256-битных хэшей), это одинаковые данные.
- Хотя всегда существует маленький шанс, что данные всё-таки разные, несмотря на одинаковые хэши.

Проверка целостности данных

- Хэш также может использоваться для проверки целостности данных при передаче. Вы передали гигабайт данных.
- А затем передали 32-байтный хэш.
- Получатель на своей стороне захешировал этот гигабайт тем же способом (той же хэш-функцией) и получил тот же самый хэш.
- Теперь он уверен, что он имеет точно те же данные, что и отправитель

Пример с облачным сервисом

- Популярный облачный сервис заблокировал одного из своих пользователей за распространение контента, защищенного авторскими правами.
- Герой истории тут же написал об этом в одной из социальных сетей, запустив волну негодования среди пользователей сервиса, ринувшихся обвинять облачный сервис в том, что он якобы позволяет себе просматривать содержимое клиентских аккаунтов, хотя не имеет права этого делать.
- Владелец защищенного копирайтом контента имел на руках хеш-коды определенных аудио- и видеофайлов, запрещенных к распространению, и занес их в черный список хешей. Когда пользователь предпринял попытку незаконно распространить некий контент, автоматические сканеры облачного сервиса засекли файлы, чьи хеши оказались в пресловутом списке, и заблокировали возможность их распространения.

Хранение паролей

Общая схема процесса регистрации аккаунта и аутентификации в системе учетных записей, основанной на хешировании, выглядит следующим образом:

1. Пользователь создает учетную запись;
2. Вычисляется хеш-код от пароля и сохраняется в базе данных. Нет никакого смысла в пароле, записанном в обычном текстовом (незашифрованном) формате на жесткий диск;
3. Когда пользователь пытается войти в систему, хеш-код, вычисленный от пароля, который он ввел, сравнивается с хеш-кодом реального пароля (извлеченным из базы данных);
4. Если хеш-коды совпадают, пользователю предоставляется доступ. Если нет – пользователю сообщается, что он ввел неверные данные для входа в учетную запись;
5. Шаги 3 и 4 повторяются каждый раз, когда кто-либо пытается войти в свой аккаунт.

Моральная ответственность

- **Если вы каким-либо образом храните чьи-то пароли, вы несете за них ответственность.**
- Каждый раз, когда кто-нибудь взламывает базу данных и раскрывает пароли, хранящиеся в открытом виде, или слабые хэши, все пострадавшие пользователи должны не только поменять свои пароли к каждому сервису, для которого использовался раскрытый пароль, но и задуматься, хотят ли они продолжать использовать взломанный сервис.

- Атаки по словарю

```
1 Trying apple      : fail
2 Trying blueberry  : fail
3 Trying 1234567     : fail
4 .....
5 Trying letmein    : fail
6 Trying qwerty     : Success!
```

- Атаки полным перебором

```
1 Trying aaaa       : fail
2 Trying aaab       : fail
3 Trying aaac       : fail
4 Trying aaad       : fail
5 .....
6 Trying acdb       : fail
7 Trying acdc       : Success!
```

Солим пароли

- Представим простую авторизацию. От пользователя к нам приходит связка значений логин/пароль, мы получаем хеш пароля и сравниваем данную связку с данными, хранящимися в базе. Для простоты будем использовать MD5 и примеры кода на PHP.

```
$password = md5($password);
```

- В данном случае, если у пользователя пароль **qwerty**, мы получим следующий хеш: d8578edf8458ce06fbc5bb76a58c5ca4. Если злоумышленник получит доступ к нашей базе, для подбора паролей он может воспользоваться уже готовыми сервисами(<http://wordd.org/D8578EDF8458CE06FBC5BB76A58C5CA4>), в которых уже есть значения, дающие данный хеш

Солим пароли

- Для защиты от уже готовых таблиц хешей с значениями, можно использовать статическую соль:

```
$password = md5($password . "MyUniqueSault");
```

- Сейчас при том же пароле qwerty мы получим совершенно другой хеш bdadb0330124cda0e8499c9cd118f7bd. Готовые таблицы уже не помогут злоумышленнику, ему придется использовать брутфорс.

Компания не хранит ваши данные

- Компания не хранит ваши данные, но очень хочет вести статистику
- Никто не мешает ей хранить хэш

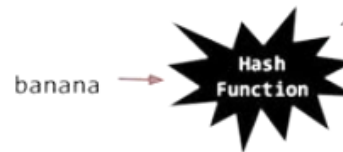
Пример

```
1  from hashlib import sha256
2
3  passwords = []
4
5  passwords.append(sha256('test'.encode()).hexdigest())
6  passwords.append(sha256('Test'.encode()).hexdigest())
7  passwords.append(sha256('test_test'.encode()).hexdigest())
8
9  enter = input("Введите пароль: ")
10
11 def check(passwords, password):
12     password = sha256(password.encode()).hexdigest()
13     for p in passwords:
14         if p == password:
15             return True
16     return False
17 print(check(passwords, enter))
18
19
```

Хэш таблицы

- **Хеш-таблица** — это структура данных для хранения пар ключей и их значений.
- По сути она представляет собой массив, где местоположение элемента зависит от значения самого элемента.
- Связь между значением элемента и его позицией в хеш-таблице задает **хеш-функция**.
- Важное свойство хеш-таблицы: поиск, вставка и удаление элементов из таблицы выполняются за фиксированное время, то есть $O(1)$, то есть они нужны тогда, когда максимально важна скорость этих операций.

Hash Function



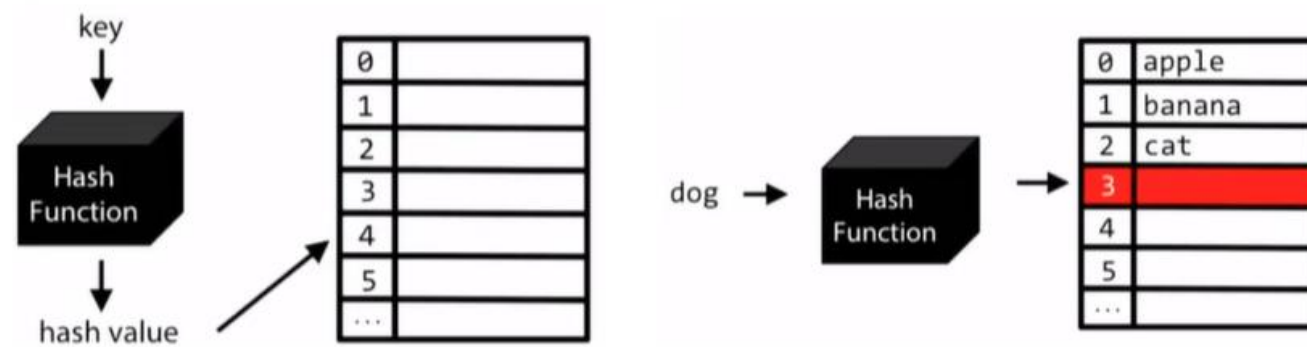
0	apple
1	
2	cantaloupe
...	
10	kiwi
...	
12	mango
...	
15	pear

Хэш таблицы

Сложность в O-символике

	В среднем	В худшем случае
Расход памяти	$O(n)$	$O(n)$
Поиск	$O(1)$	$O(n)$
Вставка	$O(1)$	$O(n)$
Удаление	$O(1)$	$O(n)$

Хэш-таблицы



ПРИМЕР

```
1 def hash(string):
2     l = len(string)
3
4     return l % 6
5
6 print(hash('asd'))
7 print(hash('231324'))
8 print(hash('dasdasdasd4as'))
9
```

Hash

0	
1	
2	
3	
4	
5	

Продолжение примера

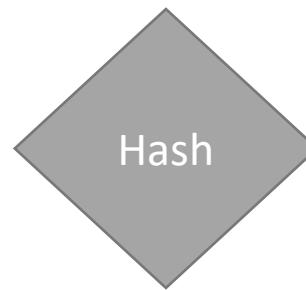
```
1 def hash(string):  
2     l = len(string)  
3  
4     return l % 6  
5  
6 print(hash('asd'))  
7 print(hash('231324'))  
8 print(hash('dasdasdasd4as'))  
9
```

Hash

0	231324
1	
2	
3	asd
4	dasdasdasd4as
5	

Продолжение примера

```
1 def hash(string):  
2     l = len(string)  
3  
4     return l % 6  
5  
6 a = {"key": "asd",  
7     "data": "Тестирование"}  
8  
9  
10  
11 print(hash(a['key']))  
12
```



0	
1	
2	
3	{"key": "asd", "data": "Тестирование"}
4	
5	

Продолжение примера

```

1  def hash(string):
2      l = len(string)
3
4      return l % 6
5
6  a = {"key": "asd",
7       "data": "Тестирование1"}
8
9  b = {"key": "231234",
10       "data": "Тестирование2"}
11
12  c = {"key": "dasdasdasd4as",
13       "data": "Тестирование3"}
14
15
16
17  print(hash(a['key']))
18  print(hash(b['key']))
19  print(hash(c['key']))
20
21

```

0	"data": "Тестирование2"	231234
1		
2		
3	"data": "Тестирование"	asd
4	"data": "Тестирование3"	dasdas dasdas d4as
5		

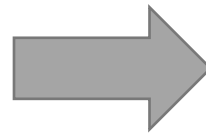
Продолжение примера

```
1 def hash(string):
2     l = len(string)
3
4     return l % 6
5
6 a = {"key": "asd",
7      "data": "Тестирование1"}
8
9 b = {"key": "231234",
10      "data": "Тестирование2"}
11
12 c = {"key": "dasdasdasd4as",
13      "data": "Тестирование3"}
14
15
16
17 print(hash(a['key']))
18 print(hash(b['key']))
19 print(hash(c['key']))
20
21
```

```
1 def hash(string):
2     r = 0
3     for i in string:
4         print(i, ord(i))
5         r += ord(i)
6
7
8     return r % 6
9
10 a = {"key": "asd",
11      "data": "Тестирование1"}
12
13 b = {"key": "231234",
14      "data": "Тестирование2"}
15
16 c = {"key": "dasdasdasd4as",
17      "data": "Тестирование3"}
18
19
20
21 print(hash(a['key']))
22 print(hash(b['key']))
23 print(hash(c['key']))
24
25
```

Продолжение примера

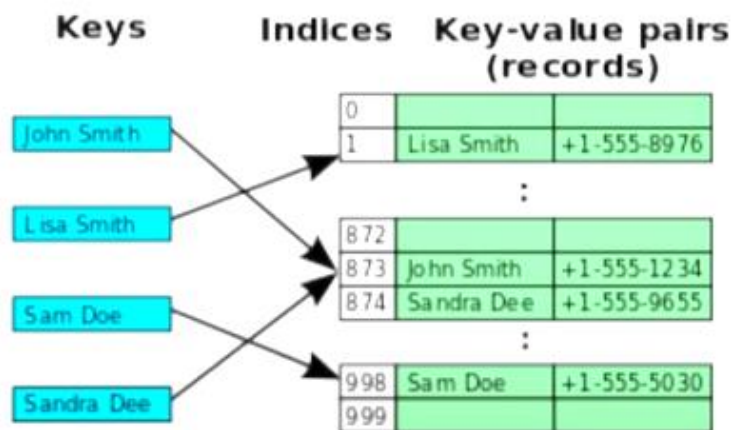
```
1  def hash(string):
2      r = 0
3      for i in string:
4          print(i, ord(i))
5          r += ord(i)
6
7
8      return r % 6
9
10 a = {"key": "asd",
11      "data": "Тестирование1"}
12
13 b = {"key": "231234",
14      "data": "Тестирование2"}
15
16 c = {"key": "dasdasdasdasd4as",
17      "data": "Тестирование3"}
18
19
20
21 print(hash(a['key']))
22 print(hash(b['key']))
23 print(hash(c['key']))
24
25
```



```
a 97
s 115
d 100
0
```

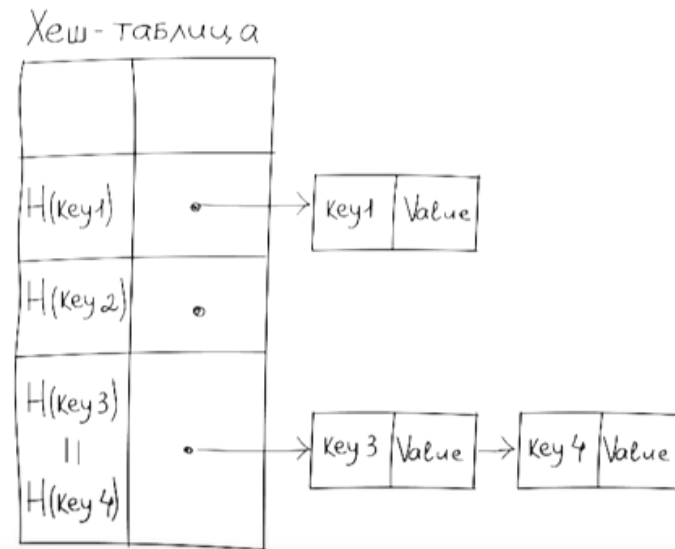
Коллизии в хэш-таблице

- Коллизия между записями «John Smith» и «Sandra Dee» (обе записи имеют хеш-значение 873) разрешается помещением записи «Sandra Dee» на следующее свободное место, ячейку 874.

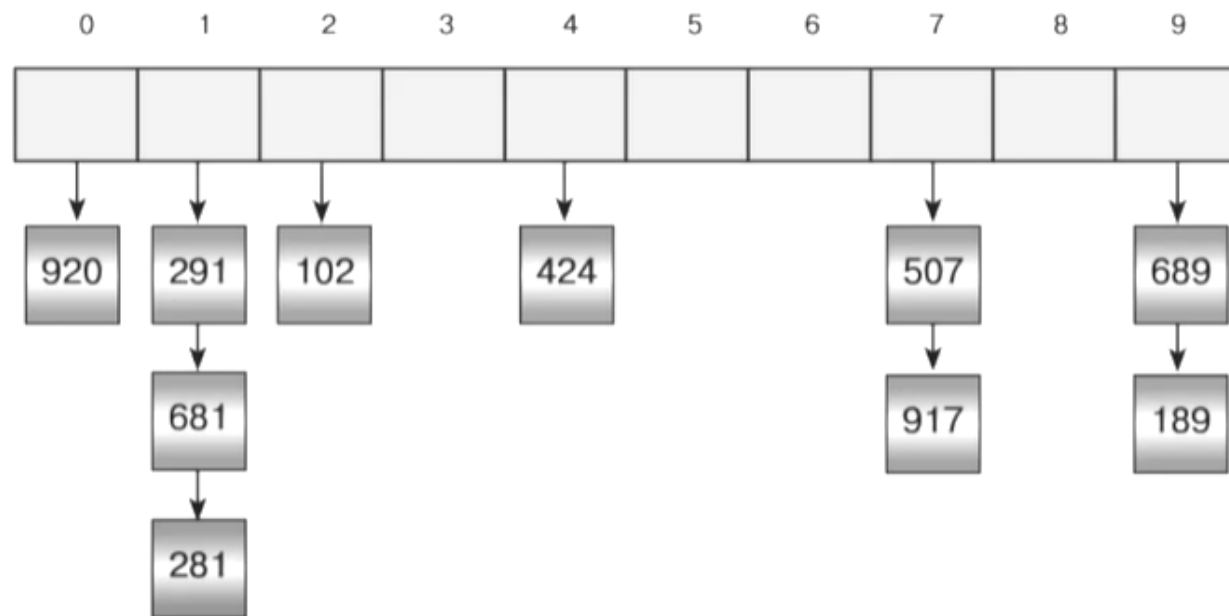


Методы разрешения коллизий (метод цепочек)

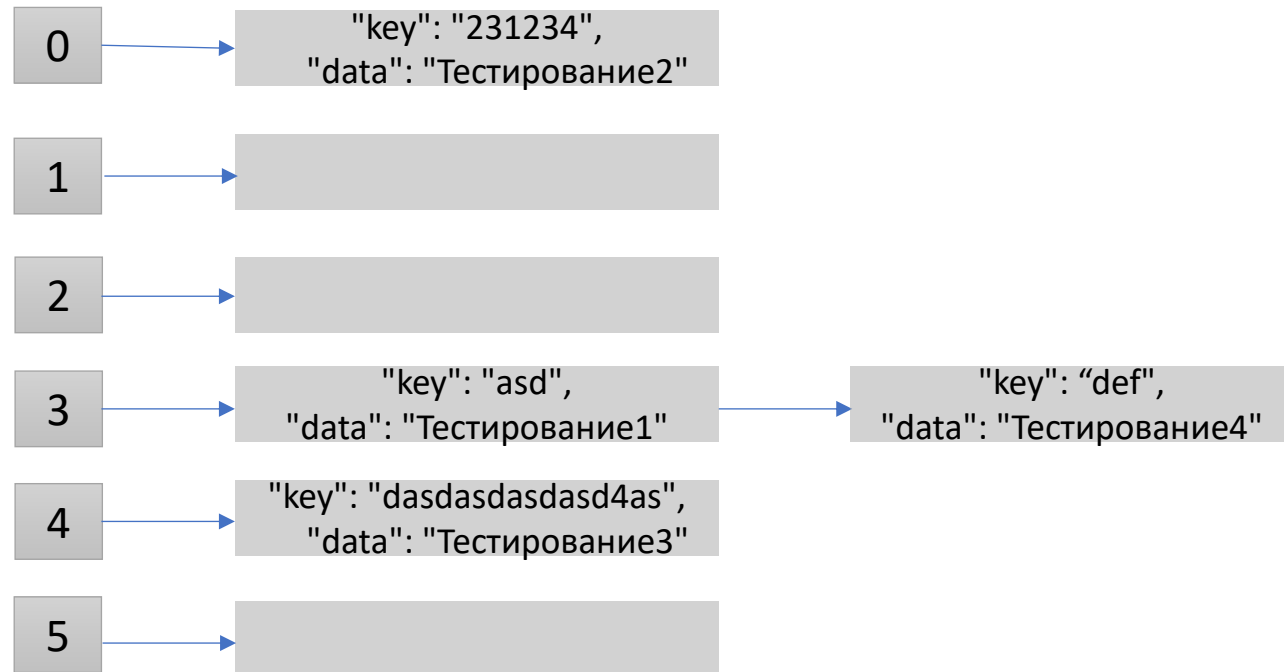
- Этот метод часто называют открытым хешированием. Его суть проста — элементы с одинаковым хешем попадают в одну ячейку в виде связанного списка.



Метод цепочек



Пример

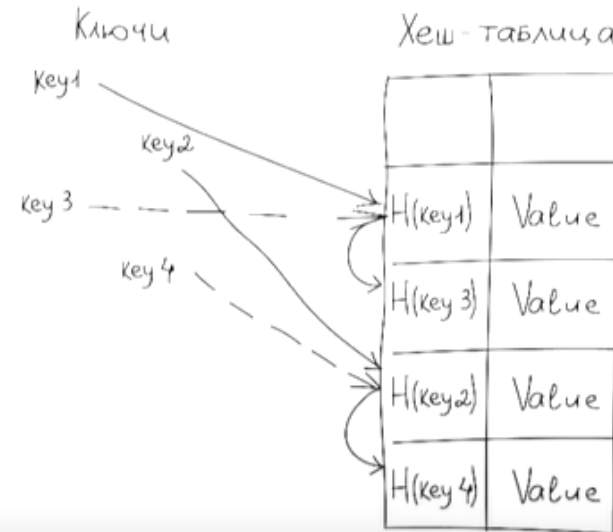


Пример

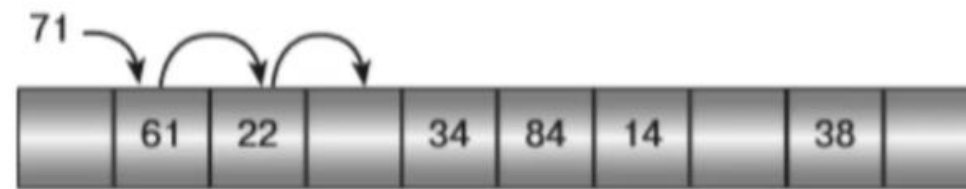


Методы разрешения коллизий (открытая индексация)

- Это значит, что пары ключ-значение хранятся непосредственно в хеш-таблице. А алгоритм вставки проверяет ячейки в некотором порядке, пока не будет найдена пустая ячейка. Порядок вычисляется на лету.



Линейное пробирование



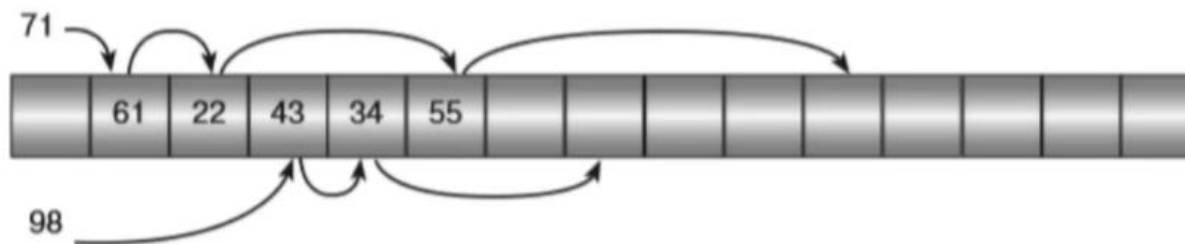
Пример

0	"data": "Тестирование2"	231234	0
1			
2			
3	"data": "Тестирование"	asd	3
4	"data": "Тестирование3"	dasdasdas dasd4as	4
5	"data": "Тестирование4"	def	3

Пример

```
1 m = [1,3,4,0,23,0,9,0]
2
3 i = 0
4
5 for item in range(i,len(m)):
6     print(m[item])
7     if m[item] ==0:
8         print(item, 'NULL')
9         break
```

Квадратичное пробирование



Удаление из хэш-таблицы

0	Eric Swanson	e.swanson	0
1	Amalia Plourde	a.plourde	0
2	Toreg Olafsson	t.olafsson	2
3	Kevin Jones	kevin.j	3
4	Karthik Gupta	k.gupta	1
5			
6	Clark Duggan	cl.duggan	6

0	Eric Swanson	e.swanson	0
1	Amalia Plourde	a.plourde	0
2			
3	Kevin Jones	kevin.j	3
4	Karthik Gupta	k.gupta	1
5			
6	Clark Duggan	cl.duggan	6

0	Eric Swanson	e.swanson	0
1	Amalia Plourde	a.plourde	0
2	Karthik Gupta	k.gupta	1
3	Kevin Jones	kevin.j	3
4			
5			
6	Clark Duggan	cl.duggan	6

Пример

0	"data": "Тестирование2"	231234	0
1			
2			
3	"data": "Тестирование"	asd	3
4	"data": "Тестирование3"	dasdasdas dasd4as	4
5	"data": "Тестирование4"	def	3

Пример

0	"data": "Тестирование2"	231234	0
1			
2			
3	"data": "Тестирование4"	def	3
4	"data": "Тестирование3"	dasdasdas dasd4as	4
5			

Пример

0	"data": "Тестирование 2"	231324	0
1			
2	"data": "Тестирование 1"	b	1
3	"data": "Тестирование 1"	c	1
4	"data": "Тестирование 3dasdasdasdasd4as"		4
5	"data": "Тестирование 1"	def1	4
6	"data": "Тестирование 1"	def	3

Размерность

- Обычно хэш-таблица содержит 50% пустых значений, чтобы избегать коллизий и быстро осуществлять поиск.
- Размерность в хэш-таблице - это количество уникальных ключей, которые могут быть сохранены в хэш-таблице.
- Размерность в хэш-таблице необходима для определения максимального количества уникальных ключей, которые могут быть сохранены в хэш-таблице. Это позволяет эффективно использовать память и ускорить поиск и вставку элементов в хэш-таблицу.

Вопросы

